LLM-based Agent Optimization for CUDA Benchmarks: Capabilities, Limitations, and Verification Challenges

Final Project — Parallel and Distributed Programming, Spring 2026

Chun-Min Chang
r14525078
NTU ESOE SDLAB
r14525078@ntu.edu.tw

Sheng-Hua Wang
r14921063
NTU EE RSALAB
r14921063@ntu.edu.tw

Yi-Hsuan Huang
r14525084
NTU ESOE SDLAB
r14525084@ntu.edu.tw

***Abstract*—This paper studies LLM-based agents for CUDA program optimization using selected HeCBench benchmarks. The goal is not only to measure whether generated code becomes faster, but also to examine whether the reported speedups are correct, reproducible, and auditable. We organize the study around an optimization workflow: prompt design, baseline execution, agent-generated modification, Slurm-based measurement, correctness checking, result classification, and human review. The experimental evidence combines prompt-constraint experiments on ten CUDA benchmarks, Phase 3 human-in-the-loop case studies, peer_data1.md and peer_data2.md, and remaining Phase 2 benchmark summaries. The results show that agents can produce credible speedups for several regular kernels, including softmax, top-k, Adam, adjacent difference, and random access. However, irregular graph algorithms, convergence-based programs, extreme speedup claims, and benchmark-aware shortcuts require careful interpretation. Strong prompt constraints and human review do not necessarily maximize the largest reported speedup, but they greatly improve the reliability of the experimental record.**

***Index Terms*—Large Language Models, LLM-based agents, CUDA optimization, GPU benchmarks, HeCBench, correctness verification**

Project Category:
☐ **A** (Self-chosen topic)
■ **B** (AI-agent code optimization)

I. Introduction

Large Language Models (LLMs) are increasingly used as code-generation and optimization agents. For CUDA programs, this capability is attractive because GPU optimization often requires tedious code transformations, repeated compilation, performance measurement, and hardware-specific reasoning. An agent can rapidly propose modifications, interpret compiler or runtime feedback, and iterate over candidate implementations. This creates an appealing workflow for parallel programming education and research: the human operator can focus on experiment design and auditing while the agent explores code-level variants.

The same setting is also risky. A CUDA program may become faster because a kernel is genuinely improved, but it may also become faster because the agent removed work, changed the timed region, weakened validation, used a broken baseline, or exploited fixed benchmark inputs. These risks are amplified on GPUs because correctness often depends on subtle synchronization, atomic operations, reduction order, memory hierarchy, and launch configuration. Therefore, a study of LLM-agent optimization should not simply rank benchmarks by speedup. It should ask whether each speedup is meaningful.

This paper uses the CUDA subset of HeCBench [1] to study this problem. The experimental purpose is threefold. First, we want to observe where LLM agents are effective on real CUDA benchmarks. Second, we want to identify why agent-generated optimizations fail. Third, we want to evaluate what workflow constraints make results more auditable. The final dataset combines four evidence sources: (1) ten prompt-constraint experiments from the main project, (2) Phase 3 agent-only and human-guided studies on softmax, top-k, and shared-memory microbenchmarks, (3) peer_data1.md and peer_data2.md report for graph, dynamic-programming, sorting, and split kernels, and (4) remaining Phase 2 summaries for Adam, adjacent difference, dropout, filter, min/max, nonzero, randomAccess, reverse, scan, and top-k.

II. Background and Motivation

Recent work has studied LLMs as code generators and optimizers. Survey work summarizes the rapid progress of LLM-based code generation [2]; OPRO formulates optimization itself as a prompting process [3]; and search-based LLM code optimization treats optimization as iterative refinement rather than one-shot generation [4]. LLM-agent studies further show that tool use and execution feedback are powerful, but also that agents still struggle with planning, long-horizon instruction following, and reliable self-evaluation [5]. Self-debugging can repair programs, but execution feedback can be incomplete or biased [6].

CUDA optimization is a useful stress test for these systems. A successful CUDA optimization often requires hardware-

aware reasoning about memory coalescing, shared memory, warp behavior, occupancy, synchronization, launch overhead, and host-device transfer. At the same time, benchmarks contain measurement and validation code that an agent may unintentionally or intentionally exploit. This motivates an evaluation style that separates different result meanings: *kernel optimization*, *parameter tuning*, *environment fix*, *measurement fix*, *measurement-equivalent change*, and *benchmark-aware optimization*. The last category is especially important in this project. Based on available summaries, some large speedups appear to use fixed input generation, repeated work, or validation structure. We report these neutrally: they show that the agent can reason about benchmark structure, but they are weaker evidence for general CUDA kernel optimization.

III. Workflow and Methodology

A. Research Questions

The study is organized around seven questions: (RQ1) Can LLM agents produce effective CUDA optimizations? (RQ2) Which benchmark classes succeed or fail? (RQ3) Are reported speedups trustworthy? (RQ4) Do prompt constraints reduce pseudo-speedups? (RQ5) Can human review turn partial optimizations into valid strategies? (RQ6) Which results are kernel optimizations versus measurement, environment, or benchmark-aware effects? (RQ7) Does evidence-guided aggressive optimization improve upon human-guided baselines?

B. Benchmark Scope and Platform

Table I lists the 29 HeCBench CUDA benchmarks used in this study. All available execution records use the same hardware setting: NVIDIA Tesla V100-SXM2-32GB, CUDA 12.8 / nvcc V12.8.61, target architecture `sm_70`, and a Slurm-based GPU node.

TABLE I
Selected CUDA Benchmarks Used in This Study

Category	Benchmark
Graph Algorithms	cc-cuda
Graph Algorithms	gc-cuda
Graph Algorithms	mis-cuda
Graph Algorithms	adjacent-cuda
Dynamic Programming	floydwarshall-cuda
Dynamic Programming	floydwarshall2-cuda
Sorting and Selection	merge-cuda
Sorting and Selection	quicksort-cuda
Sorting and Selection	sortkv-cuda
Sorting and Selection	bitonic-sort-cuda
Sorting and Selection	topk-cuda
Array and Tensor Operations	filter-cuda
Array and Tensor Operations	minmax-cuda
Array and Tensor Operations	nonzero-cuda
Array and Tensor Operations	reverse-cuda
Array and Tensor Operations	scan-cuda
Array and Tensor Operations	split-cuda
Machine Learning Kernels	adam-cuda
Machine Learning Kernels	softmax-cuda
Machine Learning Kernels	dropout-cuda
Machine Learning Kernels	moe-cuda
Machine Learning Kernels	moe-align-cuda
Machine Learning Kernels	prefetch-cuda
Communication and Memory Bandwidth	shmembench-cuda
Communication and Memory Bandwidth	randomAccess-cuda
Communication and Memory Bandwidth	p2p-cuda
Communication and Memory Bandwidth	allreduce-cuda
Communication and Memory Bandwidth	pingpong-cuda
Communication and Memory Bandwidth	simpleMultiDevice-cuda

C. Optimization Workflow

The central idea is that each agent output is treated as a hypothesis, not as a final optimization. The workflow first establishes a baseline, then applies an agent-generated change, then validates correctness and performance, and finally classifies the outcome. This framing follows the detailed project report: the important unit of analysis is not only a generated patch, but also the measurement context that makes the patch interpretable.

The project uses several prompt styles. peer_data1.md and peer_data2.md used a general prompt requesting complete optimized code with identical output, and an environment-aware prompt adding V100, `sm_70`, and Slurm. The main project used three prompt levels. P1 was weak and did not enforce measured baselines or structured logs. P2 required baseline measurement, raw-output preservation, attempt limits, and an agent summary. P3 added correctness gates, repeated trials, standardized CSV rows, profiler notes when available, contradiction checks, and result-type classification.

Phase 3 added human-in-the-loop control. Mode A measured agent-only baselines and exposed measurement noise. Mode B required robust baselines and human approval before optimization rounds. Mode C allowed more aggressive transformations, but only with paired timing, correctness checks, profiler evidence, and final confirmation.

The stages have different experimental purposes. Phase 1 is primarily diagnostic: it maps benchmarks to expected optimization space and expected failure modes. For example, `softmax-cuda` and `topk-cuda` are treated as AI primitive or kernel-optimization candidates, while `allreduce-cuda` and `pingpong-cuda` are treated as communication or measurement-sensitive cases. Phase 2 uses prompt strength as the controlled variable. It asks whether the agent becomes more reliable when the prompt demands a baseline, raw-output preservation, correctness evidence, and structured reporting. Phase 3 changes the question again: the agent is no longer evaluated as an autonomous one-shot optimizer, but as part of a human-governed workflow in which partial candidates may be rejected, restricted to certain input shapes, or promoted only after final confirmation.

This distinction matters because the same numerical speedup can mean different things in different stages. A one-shot P1 speedup without a measured baseline is only a hypothesis. A P3 speedup with repeated trials and contradiction checks is stronger evidence. A Mode B or Mode C speedup is stronger still when it is paired against an accepted baseline and survives human review. Conversely, a result can be useful even without being a kernel speedup: `allreduce-cuda` revealed environment and launcher con-

TABLE II
PROMPT-LEVEL SUMMARY FROM THE TEN-BENCHMARK STUDY.

Level	N	Mean	Median	CSV	Bad base
P1	10	1.552x	1.128x	0	4
P2	10	1.369x	1.117x	0	0
P3	10	1.131x	1.097x	10	2

straints, while `pingpong-cuda` showed how measurement setup can dominate the apparent result.

D. Auditing Rules

The following rules were applied during analysis. A result with correctness failure is invalid regardless of speedup. A missing or invalid baseline produces no speedup claim. Improvements below 1% are classified as `MEASUREMENT_EQUIVALENT`. Environment or launcher repairs are `ENV_FIX`, not `KERNEL_OPT`. Changes that appear to exploit fixed benchmark structure are labeled `BENCHMARK_AWARE_OPT`. If raw logs, system prompts, diffs, or variance data are unavailable, the field is marked N/A and the evidence is treated as lower confidence.

IV. EXPERIMENTAL SETUP

Table I summarizes the benchmark families and evidence sources. The main prompt-constraint study contains ten benchmarks with P1/P2/P3 results. Files peer_data1.md and peer_data2.md provides baseline, optimized, and environment-optimized timing for ten additional programs, but lacks raw logs and source diffs. The remaining Phase 2 summary adds ten programs with compact change descriptions, P1/P2/P3 speedup summaries, and result types.

For Phase 3 softmax, the official baseline was the existing optimized implementation `impl=1`; the naive-to-optimized gap was not counted as an agent speedup. For top-k, robust baselines were required because Mode A exposed high-CV pseudo-speedups. For shared-memory microbenchmarks, only `block_size=256` was treated as the official validated comparison; other block sizes were diagnostic failures.

V. RESULTS

A. Prompt-constraint Results

Prompt strength primarily improved auditability rather than raw average speedup. Table II summarizes the main prompt-level trend. P1 had the largest apparent mean speedup, but no CSV source and four missing or invalid baselines. P3 had lower apparent mean speedup, but full CSV coverage and no contradiction in the final audit. This pattern supports the interpretation that strong prompts do not simply make the agent faster; they make the experimental record harder to misread.

The three prompt levels play different roles in the experimental record. P1 is useful as exploratory search, but it is also where pseudo-speedups are easiest to create. For example, `topk-cuda` had an apparent P1 speedup of 2.9936x and `pingpong-cuda` showed an apparent 1.9990x improvement in a 1 GiB NCCL case, but the records lacked the complete CSV, variance data, accepted/rejected rationale, or valid baseline evidence needed for a strong claim. Other P1 entries such as `moe-align-cuda`, `p2p-cuda`, `prefetch-cuda`, and `simpleMultiDevice-cuda` were limited by missing baselines or changed measurement scope.

P2 is the minimum level that behaves like an engineering workflow. It usually records the baseline, the final candidate, and rejected attempts. In this level, `moe-cuda` retained a valid 1.0339x result after excluding failed or timed-out attempts; `topk-cuda` reached 1.1991x by reusing CUB radix-selection workspace and removing timed allocation overhead; and `prefetch-cuda` reached 1.6234x by separating prefetch setup from timed kernel execution. However, P2 still lacks the full P3 schema, repeated trials, profiler limitation notes, and contradiction checks, so environment repairs such as `allreduce-cuda` can still be too easily misread as kernel optimization.

P3 is best interpreted as an overclaim-suppression protocol. Its average speedup is lower than P1, but its evidence is stronger: it preserves raw outputs, records CSV rows, applies correctness gates, and makes environment or measurement fixes explicit. This is why the paper treats P3 and Phase 3 final confirmations as primary evidence, while P1 and BASIC runs are used mainly to explain the search space and failure modes.

B. Prompt Effects Across Evidence Sources

Table III combines the three available views of prompt strength. However, `peer_data2.md` shows that benchmark-aware strategies can persist across P1, P2, and P3 when the benchmark itself exposes fixed input or validation structure. Therefore, prompt strength must be paired with result-type classification; otherwise, an auditable benchmark shortcut may still be mistaken for a general kernel optimization.

The prompt comparison also clarifies why this paper does not rank methods by average speedup alone. If only the largest number is rewarded, P1 and benchmark-aware transformations would appear best. If correctness, reproducibility, and auditability are included, P3 and human-guided modes become more valuable even when their reported speedups are smaller. The practical lesson is that prompt engineering should be evaluated as experimental control, not merely as performance tuning.

Representative P3 results are shown in Table IV. The strongest audited kernel examples are `softmax-cuda` and `topk-cuda`. Communication and measurement-sensitive benchmarks are deliberately classified more conservatively: `allreduce-cuda` is an environment/launcher repair and `pingpong-cuda` is measurement recovery, not kernel optimization.

The BASIC results provide useful upper-bound exploration but should not be mixed directly with normalized Phase 2 results. For instance, `softmax-cuda` BASIC/GM reported 59.593x on `slice=784`, but that compared against a naive baseline and a specific shape. In contrast, the P3 result

TABLE III
PROMPT EFFECTS ACROSS THE MAIN STUDY, PEER_DATA1.MD, AND PEER_DATA2.MD.

Evidence source	Prompt or level	Observed effect	Interpretation
Main ten-benchmark study	P1 weak prompt	highest apparent mean speedup, 4 bad baselines, no CSV	fast to generate but weakly auditable
Main ten-benchmark study	P2 constrained prompt	lower mean speedup, measured baselines, summaries	improves denominator validity and traceability
Main ten-benchmark study	P3 audit prompt	complete CSV coverage and contradiction checks	strongest evidence quality, more conservative claims
peer_data1.md basic test	generic optimization prompt	sorting gains; graph failures; suspicious Floyd-Warshall speedup	unconstrained prompts can mix real gains and invalid changes
peer_data1.md basic test	environment-aware prompt	V100/Slurm context did not consistently improve outcomes	hardware context alone is insufficient without auditing rules
peer_data2.md Phase 2	P1/P2/P3 on kernel cases	Adam, adjacent, randomAccess remain near 2x across levels	regular kernels are robust to prompt variation
peer_data2.md Phase 2	P1/P2/P3 on benchmark-aware cases	dropout, minmax, scan, reverse, topk remain extremely large	prompt strength alone does not prevent benchmark-structure exploitation

TABLE IV
REPRESENTATIVE AUDITED P3 RESULTS FROM THE MAIN PROJECT.

Benchmark	Speedup	Classification	Note
softmax-cuda	1.4575x	KERNEL_OPT	PASS 42/42
topk-cuda	1.1995x	KERNEL_OPT	14 cases, repeated trials
moe-cuda	1.0778x	KERNEL_OPT	topk=8 measurement-equivalent
moe-align-cuda	1.1504x	PARAM_TUNE	correctness evidence limited
p2p-cuda	1.0022x	TOPOLOGY/MEASURE	below 1%
shmembench-cuda	1.0293x	PARAM_TUNE	modest valid gain
allreduce-cuda	n/a	ENV_FIX	no kernel speedup claim
pingpong-cuda	n/a	MEASURE_FIX	baseline recovery

TABLE V
INITIAL TEST REPORT FOR BASELINE, OPTIMIZED, AND ENVIRONMENT-OPTIMIZED VERSIONS

Benchmark	Baseline	Optimized	Env-Optimized
cc-cuda	0.0034	fail	fail
floydwarshall-cuda	0.107097	0.000024	0.00024
floydwarshall2-cuda	0.000851	0.098891	0.09133
gc-cuda	0.000048	0.000285	fail
mis-cuda	0.00136	0.002057	fail
merge-cuda	17.03105	13.7232	16.6688
quicksort-cuda	46.1346	45.8452	fail
sortkv-cuda	88.1414	72.9803	76.29895
bitonic-sort-cuda	70.13246	33.50338	34.68863
split-cuda	3423.724	3023.754	3569.766

of 1.4575x is measured against an existing optimized implementation under a stricter comparison policy. Similarly, topk-cuda BASIC/GM reached about 1.442x and the CG version reached about 1.326x, while the P3 result of 1.1995x is more suitable as paper evidence because it includes repeated trials, correctness preservation, and a normalized schema. The difference is methodological rather than merely numerical: maximum single-point speedup and auditable research evidence are not the same object.

C. peer_data1.md and peer_data2.md Results

Table III summarizes peer_data1.md and peer_data2.md. Sorting-like workloads improved most consistently, while graph workloads failed or regressed. The Floyd-Warshall result is treated as suspicious because a 0.107097 to 0.000024 reduction is implausible for an $O(N^3)$ algorithm without strong correctness evidence. The environment-aware prompt did not consistently improve the results: it still failed on several graph or irregular cases and regressed on split-cuda. This suggests that simply telling the agent the GPU model and scheduler is not equivalent to requiring a valid experimental protocol.

The concrete timing pattern reinforces the same interpretation. bitonic-sort-cuda improved from 70.13246 to 33.50338, roughly 2.09x; sortkv-cuda improved from 88.1414 to 72.9803, roughly 1.21x; merge-cuda improved from 17.03105 to 13.7232, roughly 1.24x; and split-cuda improved from 3423.724 to 3023.754 under the generic prompt. These are plausible for regular workloads with clearer memory-access patterns, although the absence of raw correctness logs and source diffs keeps them at lower confidence.

The negative cases are equally important. cc-cuda failed under both generic and environment-aware prompts. gc-cuda and mis-cuda produced optimized timings in the summary, but the accompanying analysis indicates graph-algorithm correctness risk. quicksort-cuda improved only from 46.1346 to 45.8452 under the generic prompt, close to measurement-equivalent, and failed under the environment-aware prompt. floydwarshall-cuda reported an approximately 4462x apparent speedup, which is more plausibly skipped work, incomplete iteration, or invalid measurement than a real algorithmic improvement. floydwarshall2-cuda regressed by over two orders of magnitude. These cases show that agent failures often damage algorithmic semantics rather than merely missing an optimiza-

tion opportunity.

D. Remaining Phase 2 Benchmarks

The remaining summaries show two different patterns. Table VI separates conventional kernel optimizations from benchmark-aware optimizations and preserves the P1/P2/P3 comparison from `peer_data2.md`. Conventional kernel optimizations include `adam-cuda`, `adjacent-cuda`, and `randomAccess-cuda`; their speedups remain stable across prompt levels, which suggests that the underlying optimization opportunities are robust. In contrast, several summaries report extremely large or zero-denominator speedups across all prompt levels. Based on the available descriptions, these appear to exploit benchmark structure such as fixed inputs, repeated work, parity of repeated operations, or precomputed validation outputs. We therefore classify them as `BENCHMARK_AWARE_OPT`. This label is not a rejection: it records an agent capability to identify benchmark-specific structure, but it is weaker evidence for general CUDA kernel optimization.

The change summaries in `peer_data2.md` make the distinction clearer. `adam-cuda` uses conventional CUDA transformations: it hoists global memory reads and writes into registers and replaces repeated `powf` calls with incremental multiplication. `adjacent-cuda` removes a runtime branch through template specialization and fuses kernels to reduce memory traffic and launch overhead. `randomAccess-cuda` increases parallelism while preserving XOR validation. These changes are consistent with ordinary kernel optimization.

The benchmark-aware cases have a different character. `minmax-cuda` reuses host-side extrema inside timed repeat loops; `nonzero-cuda` uses a host-known nonzero count; `scan-cuda` copies the CPU reference result to the device before validation; `reverse-cuda` exploits the parity of repeated reverse operations; and the `topk-cuda` summary exploits deterministic permutation-row structure. These may be valid transformations for a fixed benchmark harness, but they do not demonstrate a general CUDA kernel improvement. Their persistence across P1/P2/P3 is therefore a key result: stronger prompts alone do not prevent benchmark-aware shortcuts unless the evaluation protocol labels them separately.

E. Human-guided and Evidence-guided Softmax

Phase 3 softmax demonstrates the value of human review because it contains all three phenomena that motivated the project: a strong baseline, a partial agent candidate, and a final accepted policy. Mode A first established that `impl=0` and `impl=1` should not be confused. The gap from the naive implementation to the existing optimized implementation was not counted as an agent speedup. This prevented the later experiment from reporting an inflated improvement.

Mode B then tested an agent-generated candidate that changed both row parallelism and exponent caching. The candidate improved large slices, but it regressed at `slice=128` and failed one correctness trial at `slice=256`. This was classified as partial success rather than a valid replacement. Human review kept the useful large-slice behavior and rejected the universal replacement claim. The final Mode B solution became a shape-aware dispatch: `slice=128` and `slice=256` use the existing optimized path, while `slice=784`, `slice=1024`, and `slice=2048` use the new large-slice path.

Mode C tested whether an evidence-guided aggressive candidate could improve upon the accepted Mode B implementation. The primary metric was `impl=4` versus `impl=3`, not `impl=4` versus the original baseline. As shown in Table VII, Mode C produced additional speedup for `slice=784` and `slice=1024`, while `slice=2048` was treated as measurement-equivalent. Nsight Compute evidence was limited: `impl=4` reduced dynamic shared memory by about 0.90 KB/block, but missing memory-throughput and stall metrics prevent a causal claim.

The supporting Phase 3 cases are also informative. For `topk-cuda`, Mode A showed that a pseudo-speedup could appear from high baseline variance even without a meaningful code change: two cases showed about 1.4x apparent speedup while the baseline coefficient of variation exceeded 40%. Robust baseline measurement was therefore required before any Mode B claim. In the robust baseline, 14 official cases were measured over seven trials; all correctness checks passed, 12 cases were classified as valid, two as caution, and none as noisy. For `shmembench-cuda`, the Mode B robust baseline confirmed that only `block_size=256` was an official validated comparison, with about 13226.78 GB/s average bandwidth and about 0.42% CV. The `block_size=128` and 512 cases failed checksum, while 1024 exceeded the shared-memory limit and failed to build. These cases did not produce final Mode B optimization claims, but they justify the workflow's insistence on variance checks and official-case definitions.

VI. Discussion

A. Why Regular Kernels Are More Optimizable

The results suggest that LLM agents are most reliable on regular, highly parallel workloads with clear memory-access patterns. Softmax, top-k, Adam, adjacent difference, randomAccess, and sorting kernels expose optimization opportunities such as workspace reuse, branch elimination, loop-invariant hoisting, incremental arithmetic, and shape-aware dispatch. These transformations map naturally to CUDA performance rules that are frequently represented in training data and programming examples. They also tend to preserve a simple input-output relation, making correctness easier to validate.

B. Why Irregular Kernels Fail

Irregular graph and convergence-based programs are much riskier. Files peer_data1.md and peer_data2.md for `cc`, `gc`, and `mis` suggest that the agent may break frontier propagation, atomic updates, or convergence logic. These programs often rely on repeated relaxation, global progress conditions, or fine-grained synchronization. A local source-level optimization that appears harmless can change the order or visibility of

TABLE VI
REMAINING PHASE 2 SUMMARIES FROM PEER_DATA2.MD.

Benchmark	P1 speedup	P2 speedup	P3 speedup	Interpretation
adam-cuda	∼2.0x	∼2.0x	∼2.0x	KERNEL_OPT
adjacent-cuda	∼1.99x	∼2.0x	∼2.0x	KERNEL_OPT
randomAccess-cuda	∼2.17x	∼2.23x	∼2.21x	KERNEL_OPT
dropout-cuda	1.9e4x–2.2e5x	1.5e4x–2.2e5x	1.2e4x–2.2e5x	BENCHMARK_AWARE
filter-cuda	1.61x / 4.90x	1.53x / 4.72x	1.53x / 4.61x	BENCHMARK_AWARE
minmax-cuda	8.8e7x–9.8e7x	9.4e7x–1.3e8x	1.0e8x–1.6e8x	BENCHMARK_AWARE
nonzero-cuda	timed sections → 0	timed sections → 0	timed sections → 0	BENCHMARK_AWARE
reverse-cuda	∼2038.60x	∼1976.99x	∼1871.34x	BENCHMARK_AWARE
scan-cuda	1e4–1e6x	1e4–1e6x	1e4–1e6x	BENCHMARK_AWARE
topk-cuda	147–5220x	137–5190x	137–5260x	BENCHMARK_AWARE, separate from audited P3

TABLE VII
SOFTMAX PHASE 3 FINAL INTERPRETATION. MODE B COMPARES AGAINST IMPL=1; MODE C COMPARES AGAINST ACCEPTED IMPL=3.

Slice	Mode B dispatch	Mode B baseline ms	Mode B candidate ms	Mode B speedup	Mode C additional result
128	impl=1	0.134869	0.134574	1.002197x	not claimed; measurement-equivalent
256	impl=1	0.321793	0.321505	1.000895x	not claimed; small-slice path retained
784	impl=2	1.442716	1.036402	1.392043x	1.135540x vs impl=3
1024	impl=2	2.104045	1.238443	1.698944x	1.048740x vs impl=3
2048	impl=2	2.237452	1.672904	1.337466x	measurement-equivalent vs impl=3

updates. The Floyd-Warshall case illustrates a different failure: an extreme apparent speedup is more likely incomplete computation, skipped iterations, or invalid measurement than a real algorithmic improvement.

C. Benchmark-aware Optimization

Benchmark-aware results are important but ambiguous. In `reverse-cuda`, using the parity of repeated reverse operations is logically valid for a fixed benchmark invocation, but it does not necessarily improve a general reverse kernel. In `minmax`, `nonzero`, and `scan`, the summaries suggest that the agent may reuse information already available from host-side generation or CPU reference computation. Such behavior is not necessarily "wrong" if the benchmark specification permits it, but it changes the scientific meaning of the result. It demonstrates benchmark exploitation rather than general GPU kernel improvement. This is why the paper reports these cases separately.

D. Prompt Constraints and Human Review

Prompt constraints strongly affect scientific reliability. Weak prompts can produce impressive numbers but often lack measured baselines, raw outputs, and contradiction checks. Strong prompts reduce exaggeration by forcing the agent to preserve correctness evidence and classify results. Human review is especially useful when an optimization is partially valid: the softmax case shows that human operators can reject a universal replacement while preserving a valid shape-dependent strategy. In this sense, human involvement is not mainly manual coding; it is experimental governance.

VII. CONCLUSION

LLM-based agents can optimize CUDA programs, but their outputs require strict auditing. Credible speedups appear in regular kernels and in carefully reviewed softmax/top-k cases. Failures concentrate in irregular graph algorithms, suspicious extreme speedups, noisy measurements, and benchmark-aware shortcuts. The main lesson is that agent optimization should be evaluated as an auditable workflow, not as a single generated patch. Measured baselines, correctness gates, repeated trials, result-type classification, and human review are necessary to distinguish real kernel improvements from measurement artifacts, environment fixes, or benchmark-specific shortcuts.

PROJECT WEBSITE

The project repository is available at: GitHub Project Repository.

The prompts and agent I/O records are available at: Prompt and Agent I/O Archive.

REFERENCES

[1] Z. Jin and J. S. Vetter, "A benchmark suite for improving performance portability of the sycl programming model," in *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2023, pp. 325–327.

[2] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, "A survey on large language models for code generation," *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 2, pp. 1–72, Jan. 2026. [Online]. Available: https://doi.org/10.1145/3747588

[3] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, "Large language models as optimizers," 2024. [Online]. Available: https://arxiv.org/abs/2309.03409

[4] S. Gao, C. Gao, W. Gu, and M. Lyu, "Search-based llms for code optimization," 2024. [Online]. Available: https://arxiv.org/abs/2408.12159

[5] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "Agentbench: Evaluating llms as agents," in *International Conference on Learning Representations*, B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, Eds., vol. 2024, 2024, pp. 52 989–53 046. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/file/e9df36b21ff4ee211a8b71ee8b7e9f57-Paper-Conference.pdf

[6] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," 2023. [Online]. Available: https://arxiv.org/abs/2304.05128